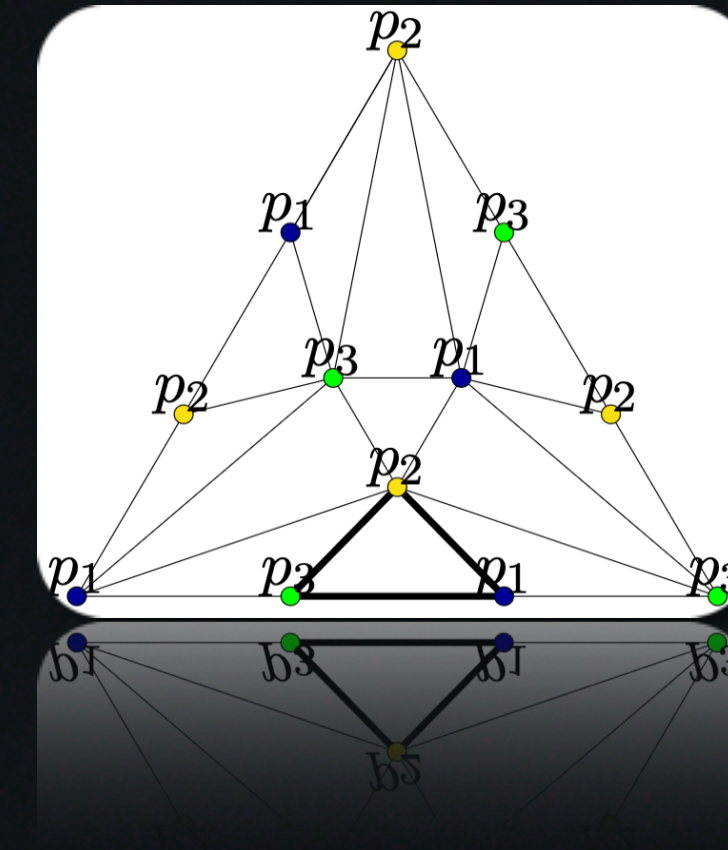


Laboratorio de Computación Concurrente

Práctica 4 Spinlocks y algunas Primitivas

INTRODUCCIÓN

- Supongamos que tenemos un contador concurrente, lanzamos 1,000 operaciones `inc()` concurrentes, usamos un Peterson Lock para garantizar MutEx.
- La corrección del Peterson Lock garantiza exclusión mutua. Al terminar de lanzar los 1,000 `inc()` el estado del contador debería ser exactamente 1,000.
- En la práctica, podemos obtener que el estado del contador final puede ser menor al esperado. Así como estudiamos en la clase pasada.
- ¿Por qué falla si teóricamente es correcto?



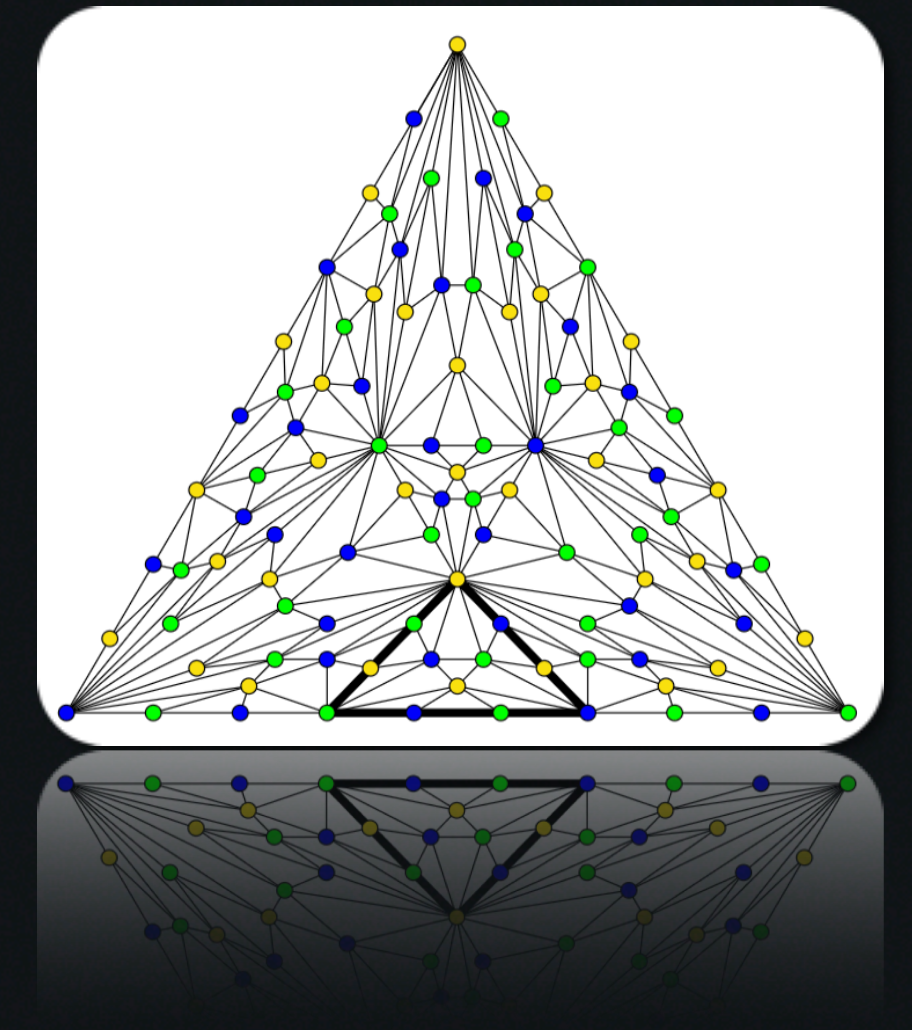
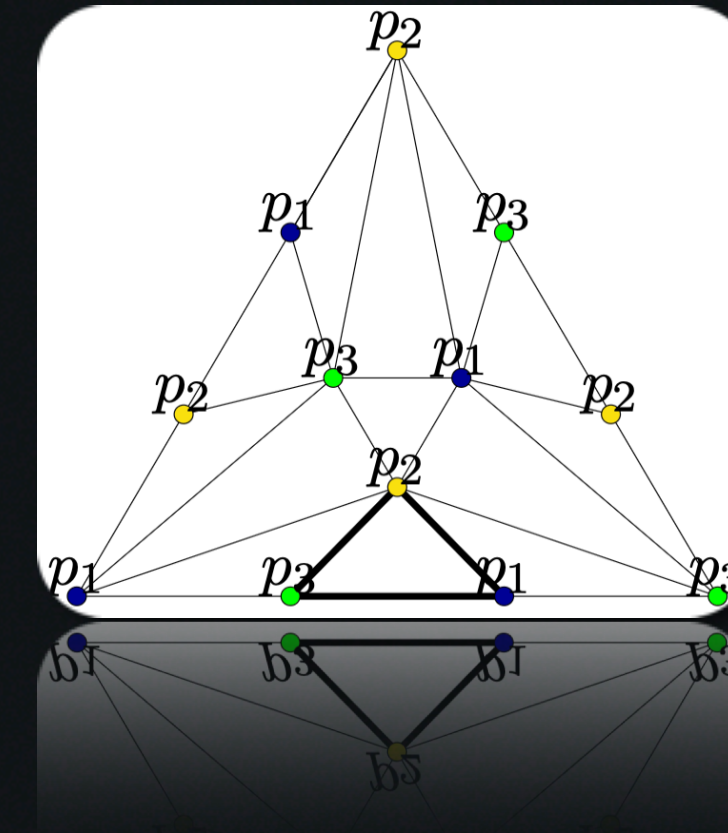
INTRODUCCIÓN

La demostración de corrección de “Peterson” asume memoria secuencialmente consistente: Cada hilo ve sus propias lecturas/escrituras en orden, y otros hilos ven estas operaciones en el mismo orden.

Como vimos en la practica anterior el orden de las operaciones entre hilos no está garantizado:

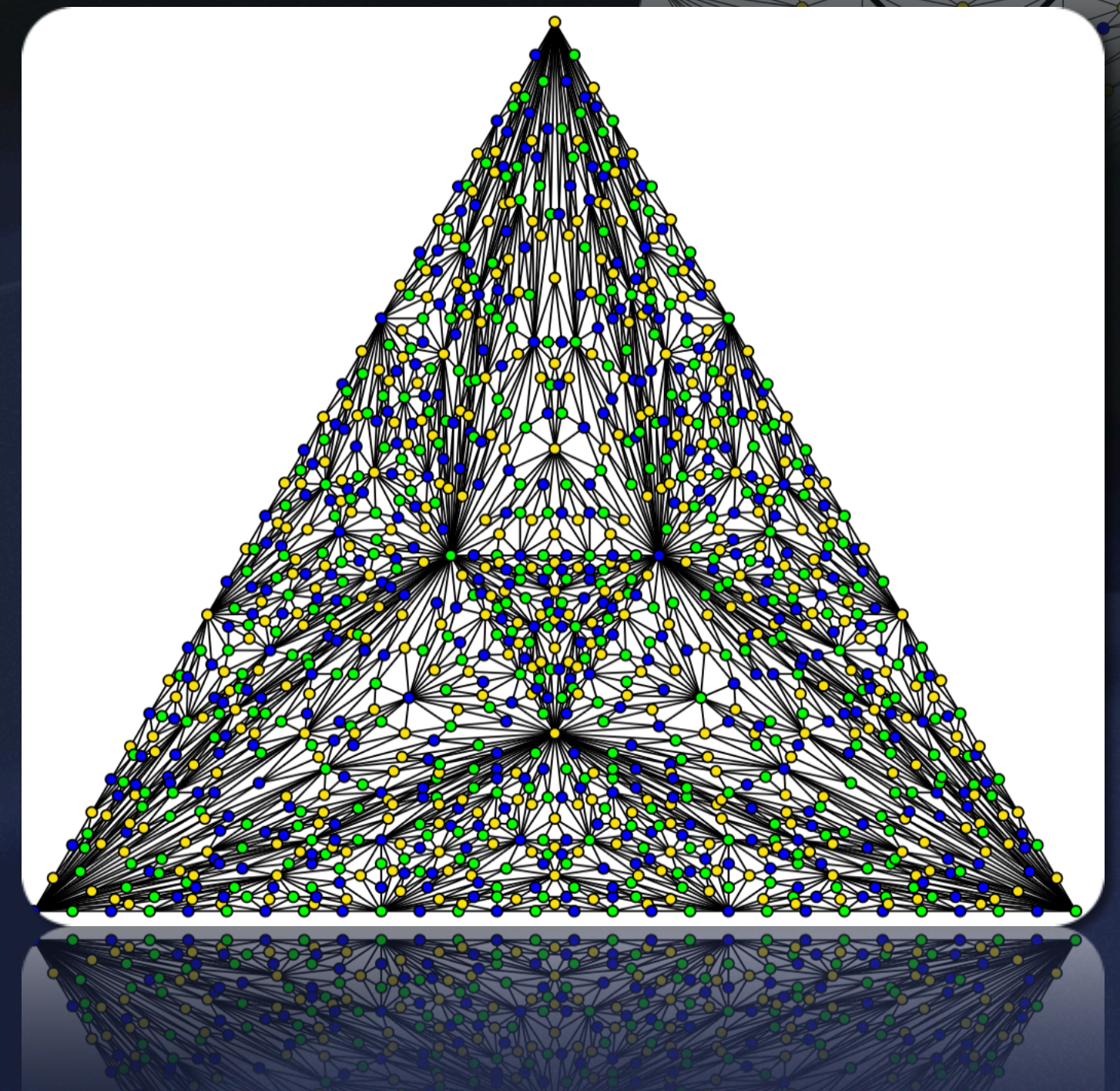
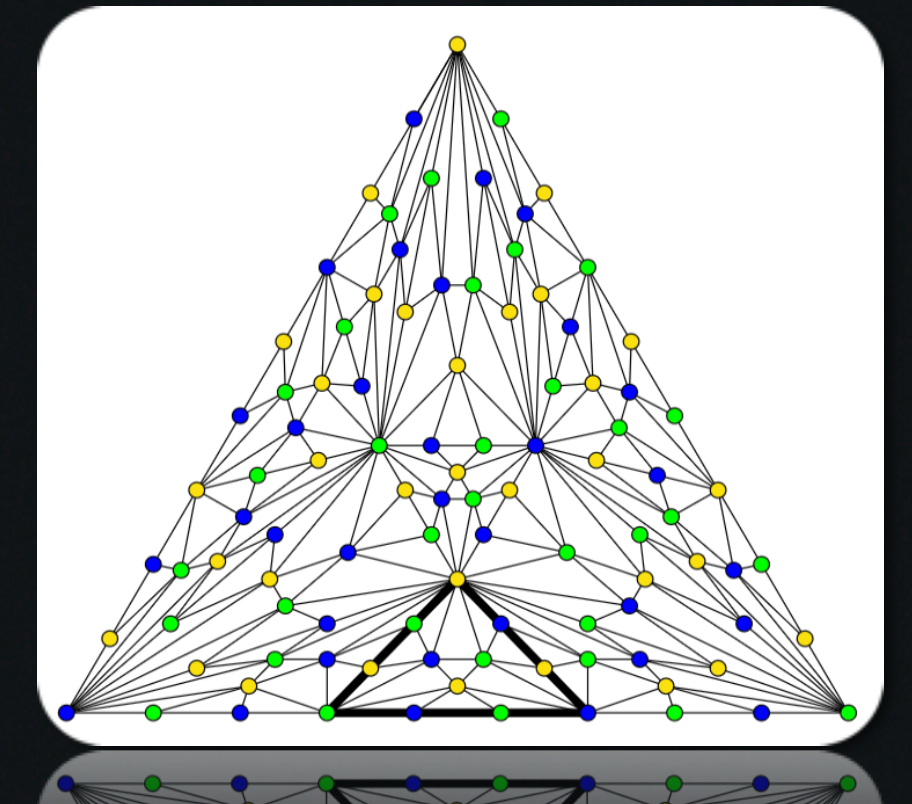
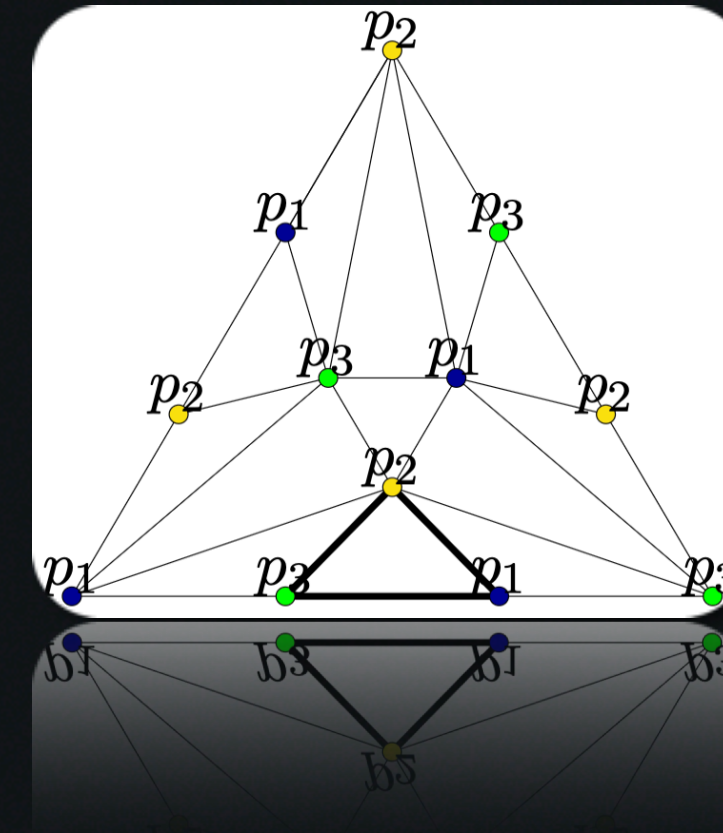
- ◆ El compilador reordena instrucciones entre variables distintas.
- ◆ Hardware multiprocesador puede retrasar la visibilidad de escrituras.

El modelo teórico no se cumple → pueden aparecer dos hilos en la sección crítica al mismo tiempo.



INTRODUCCIÓN

Entonces, cuando necesitamos consistencia secuencial debemos pedirla de manera explícita. Las arquitecturas proveen instrucciones que le indican al procesador que propague las actualizaciones hacia y desde la memoria según sea necesario, para asegurar que las lecturas y escrituras interactúen correctamente.



Write buffers y retraso de escrituras

- En arquitecturas multiprocesador, las escrituras a memoria compartida se almacenan en un `write buffer` antes de actualizar la memoria principal.
- Esto permite a la CPU seguir ejecutando instrucciones **sin esperar a que la memoria se actualice**.
- Problema: Si el hilo `A` escribe `victim` pero la escritura está en el buffer,
- Y luego lee `flag[B]`, puede **observar valores viejos**, violando la exclusión mutua.

Garantizar visibilidad y orden

- **Memory fences / barriers:**
 - Garantizan que todas las escrituras previas se apliquen antes de continuar.
 - Son costosas, porque bloquean la CPU hasta que la memoria está actualizada.
- **Operaciones atómicas:**
 - Realizan lectura y escritura de forma indivisible.
 - Aseguran que solo un hilo puede “ganar” la operación a la vez, evitando inconsistencias.
 - En la práctica: usaremos operaciones atómicas para construir candados eficientes y seguros en multiprocesadores modernos.

La Instrucción Atómica:

Test-and-Set

- **¿Qué es?**

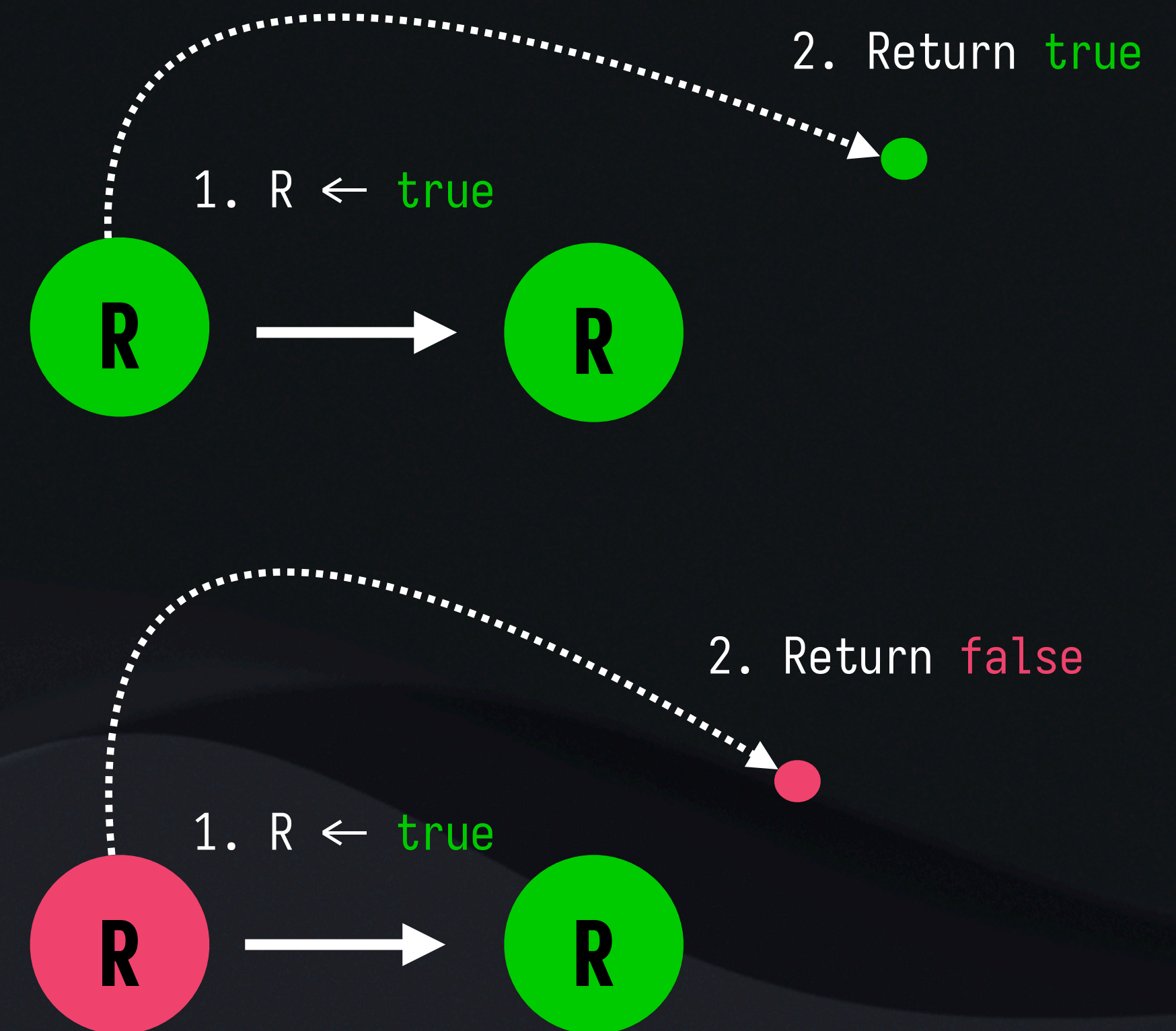
- La instrucción `TestAndSet()` es una operación atómica fundamental.
- "Atómica" significa que se ejecuta como un solo paso indivisible, sin que otros hilos puedan interferir.

- **¿Cómo funciona?**

- Opera sobre una variable booleana.
- Realiza dos acciones en un solo paso:
 1. Escribe el valor `true` en la variable.
 2. Devuelve el valor que la variable tenía **antes** de la escritura.

- **En Java:**

- Se implementa a través de la clase `java.util.concurrent.AtomicBoolean` y su método `getAndSet(true)`.



Test And Set Lock (TAS Lock)

Test-and-Set

- **La Idea Central:**

- Usaremos un AtomicBoolean llamado state para representar el estado del candado.
 - state = false: El candado está libre.
 - state = true: El candado está ocupado.

- **Método lock():**

- Para adquirir el candado, un hilo entra en un bucle.
- En cada iteración, ejecuta state.getAndSet(true).
- El bucle continúa **mientras** la operación devuelva true (indicando que el candado ya estaba ocupado).
- El hilo solo sale del bucle y adquiere el candado cuando la operación devuelve false (el estado anterior era "libre").

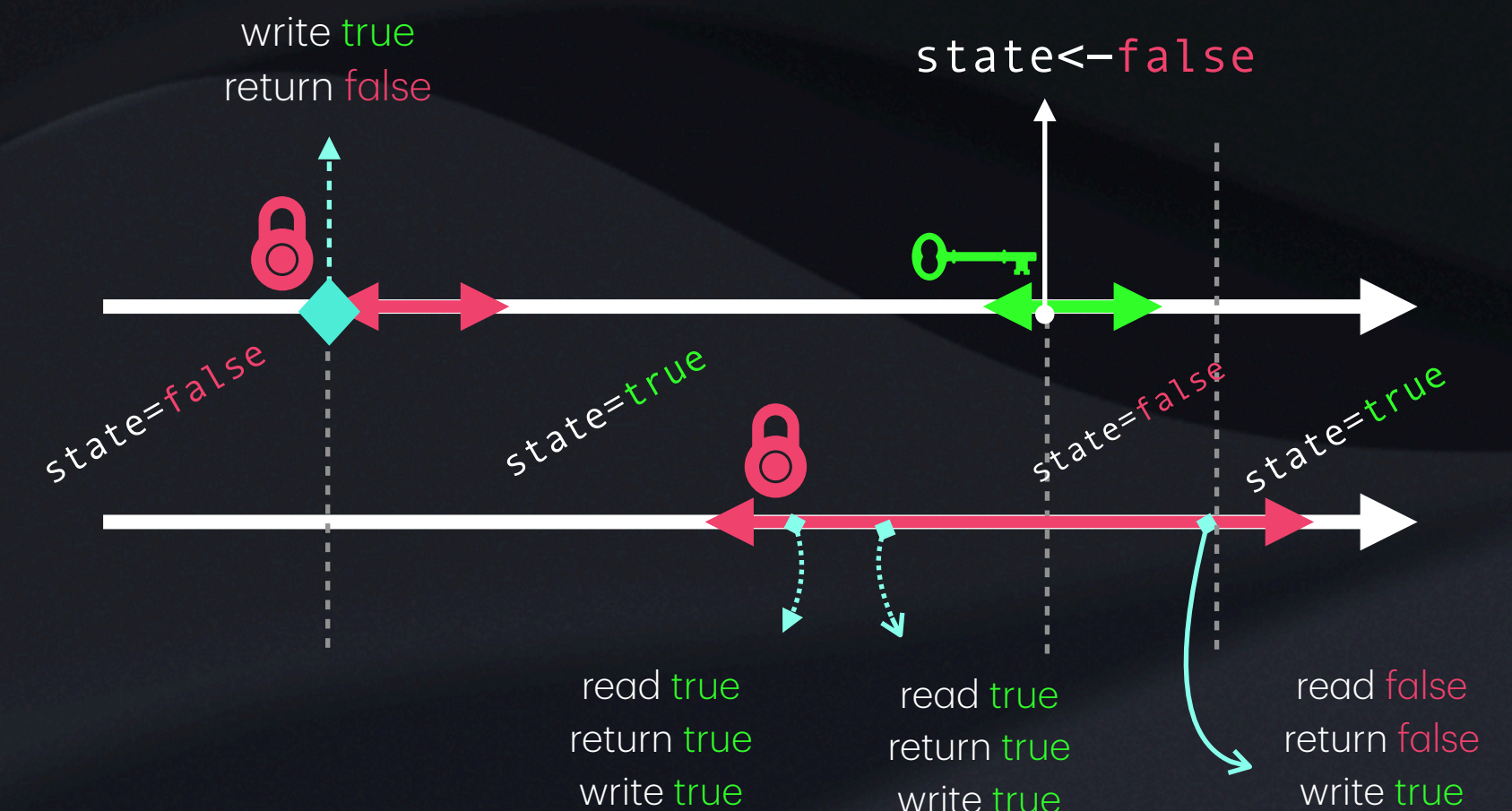
- **Método unlock():**

- Para liberar el candado, simplemente se establece el estado a false con state.set(false).

```
public class TASLock implements Lock {
    AtomicBoolean state = new AtomicBoolean(false);

    public void lock() {
        // Gira (spins) hasta que logre cambiar el estado de false a true.
        while (state.getAndSet(true)) {
            // No hace nada, solo sigue intentando.
        }
    }

    public void unlock() { state.set(false); }
}
```



Análisis de TASLock:

- **Ventajas:**
 - **Simplicidad:** Es una de las implementaciones de `spin lock` más sencillas de entender y escribir.
 - **Corrección:** Garantiza la exclusión mutua de forma correcta.
- **Desventaja: Pésimo Rendimiento bajo Contención**
 - Cuando múltiples hilos intentan adquirir el candado al mismo tiempo, el rendimiento del sistema se degrada drásticamente.
- **¿Por qué tiene un mal rendimiento?**
 - El bucle `while(state.getAndSet(true))` es una operación de escritura en memoria en cada intento.
 - En arquitecturas multiprocesador, una escritura a una dirección de memoria compartida obliga a invalidar la caché de todos los demás procesadores.
 - Esto genera un tráfico masivo en el bus de memoria, ya que todos los procesadores deben actualizar sus cachés constantemente. El bus se satura y se convierte en un cuello de botella.

Test-and-Test-and-Set (TTASLock)

- **La Idea Clave:**

- Evitar la escritura (getAndSet) mientras el candado esté visiblemente ocupado. Primero leer, y solo después intentar escribir.

- **El Proceso de lock() en TTASLock:**

- **Primer Bucle (Test):** El hilo primero "gira" en un bucle de solo lectura (while (state.get()) {}).
 - Esta lectura se satisface desde la caché local del procesador, lo cual es muy rápido y no genera tráfico en el bus.
- **Segundo Paso (Test-and-Set):** Únicamente cuando el hilo ve que state es false, sale del primer bucle e intenta adquirir el candado con la operación de escritura state.getAndSet(true).

```
public class TTASLock implements Lock {
    AtomicBoolean state = new AtomicBoolean(false);

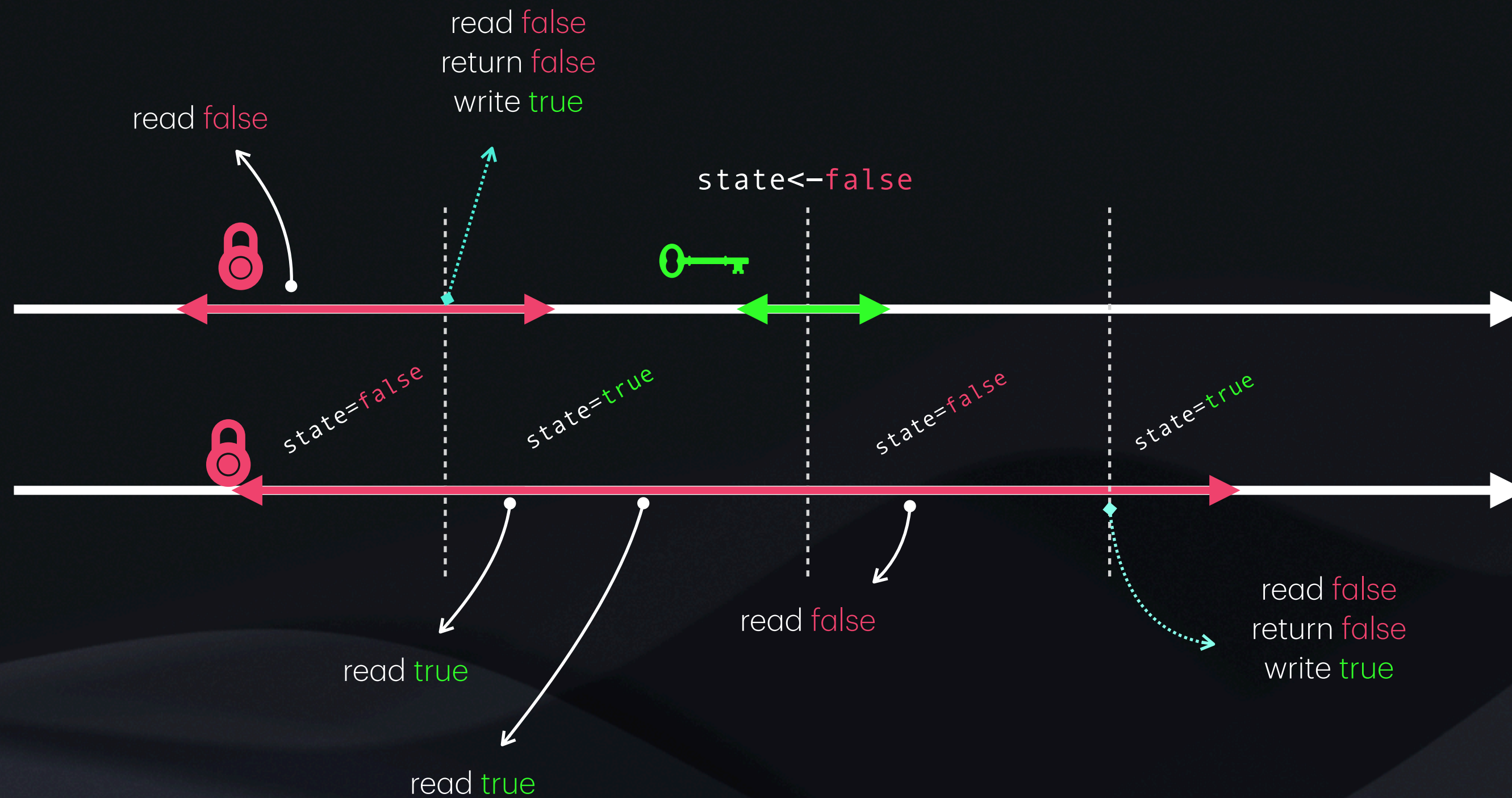
    public void lock() {

        while (true) {
            // 1. Gira localmente leyendo de la caché hasta que el lock parezca libre.
            while (state.get()) {}

            // 2. Solo entonces, intenta la escritura costosa.
            If (!state.getAndSet(true)) {
                return; // ¡Lock adquirido!
            }
        }
    }

    public void unlock() { state.set(false); } }
```


Test-and-Test-and-Set (TTASLock)



```
public class TASLock implements Lock {
    AtomicBoolean state = new AtomicBoolean(false);

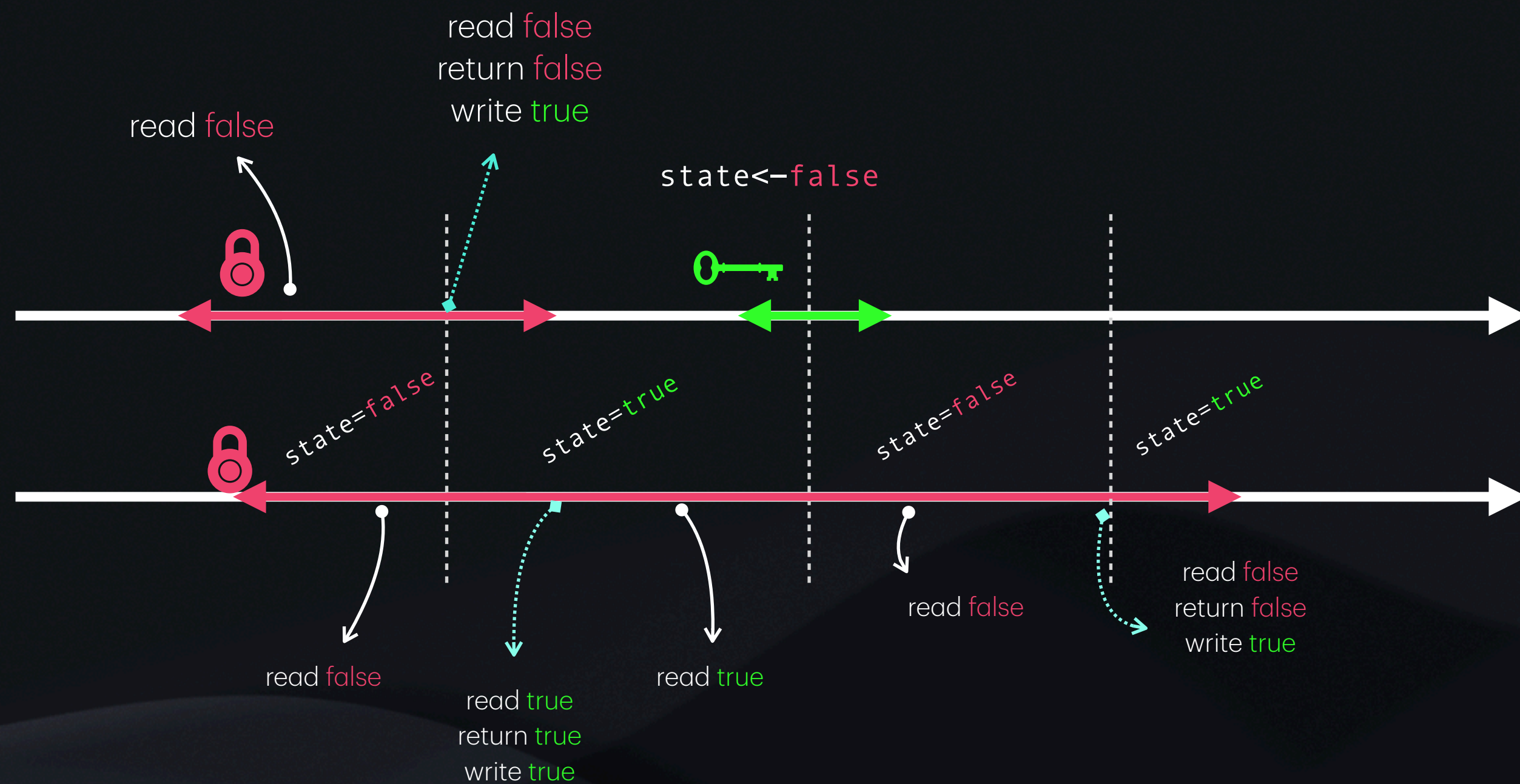
    public void lock() {
        while (true) {
            // 1. Gira localmente leyendo de la caché hasta que el lock parezca libre.
            while (state.get()) {}

            // 2. Solo entonces, intenta la escritura costosa.
            If (!state.getAndSet(true)) {
                return; // ¡Lock adquirido!
            }
        }
    }

    public void unlock() { state.set(false); } } }
```

Ejecución con las primeras dos lecturas “secuenciales”.

Test-and-Test-and-Set (TTASLock)



```
public class TASLock implements Lock {
    AtomicBoolean state = new AtomicBoolean(false);

    public void lock() {
        while (true) {
            // 1. Gira localmente leyendo de la caché hasta que el lock parezca libre.
            while (state.get()) {}

            // 2. Solo entonces, intenta la escritura costosa.
            If (!state.getAndSet(true)) {
                return; // ¡Lock adquirido!
            }
        }
    }

    public void unlock() { state.set(false); } }
}
```

Ejecución con las primeras dos lecturas “concurrentes”.

Test-and-Test-and-Set (TTASLock)

“... if some other thread acquires the lock between the first and second step, then, most likely, there is high contention for that lock”

- Cuando hay alta contención es mala idea intentar adquirir el candado ya que solo contribuiremos a una mayor contención.
- El TTASLock, bajo alta contención, degrada su rendimiento.
- Introducir una espera inteligente (backoff) puede aliviar la congestión y mejorar la eficiencia.



Exponential Backoff (BackoffLock)

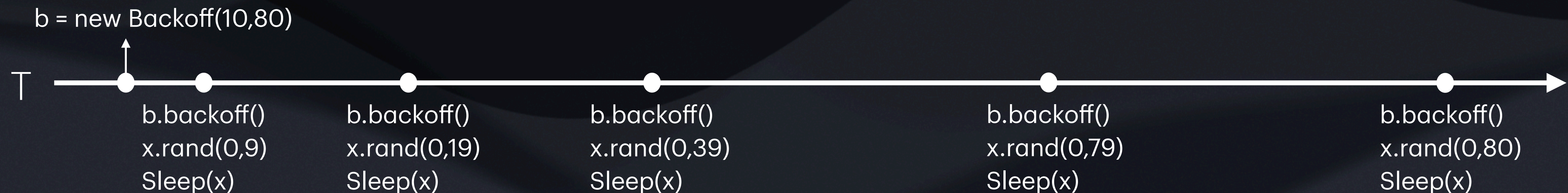
- Cuando un hilo detecta que falló al adquirir un candado recién liberado, **eso indica** alta contención.
- En lugar de insistir inmediatamente, conviene **esperar un tiempo aleatorio** antes de reintentar.
- Esta espera (backoff) reduce el tráfico y da oportunidad a otros hilos.

```
1 public class Backoff {
2     final int minDelay, maxDelay;
3     int limit;
4     final Random random;
5     public Backoff(int min, int max) {
6         minDelay = min;
7         maxDelay = min;
8         limit = minDelay;
9         random = new Random();
10    }
11    public void backoff() throws InterruptedException {
12        int delay = random.nextInt(limit);
13        limit = Math.min(maxDelay, 2 * limit);
14        Thread.sleep(delay);
15    }
16 }
```

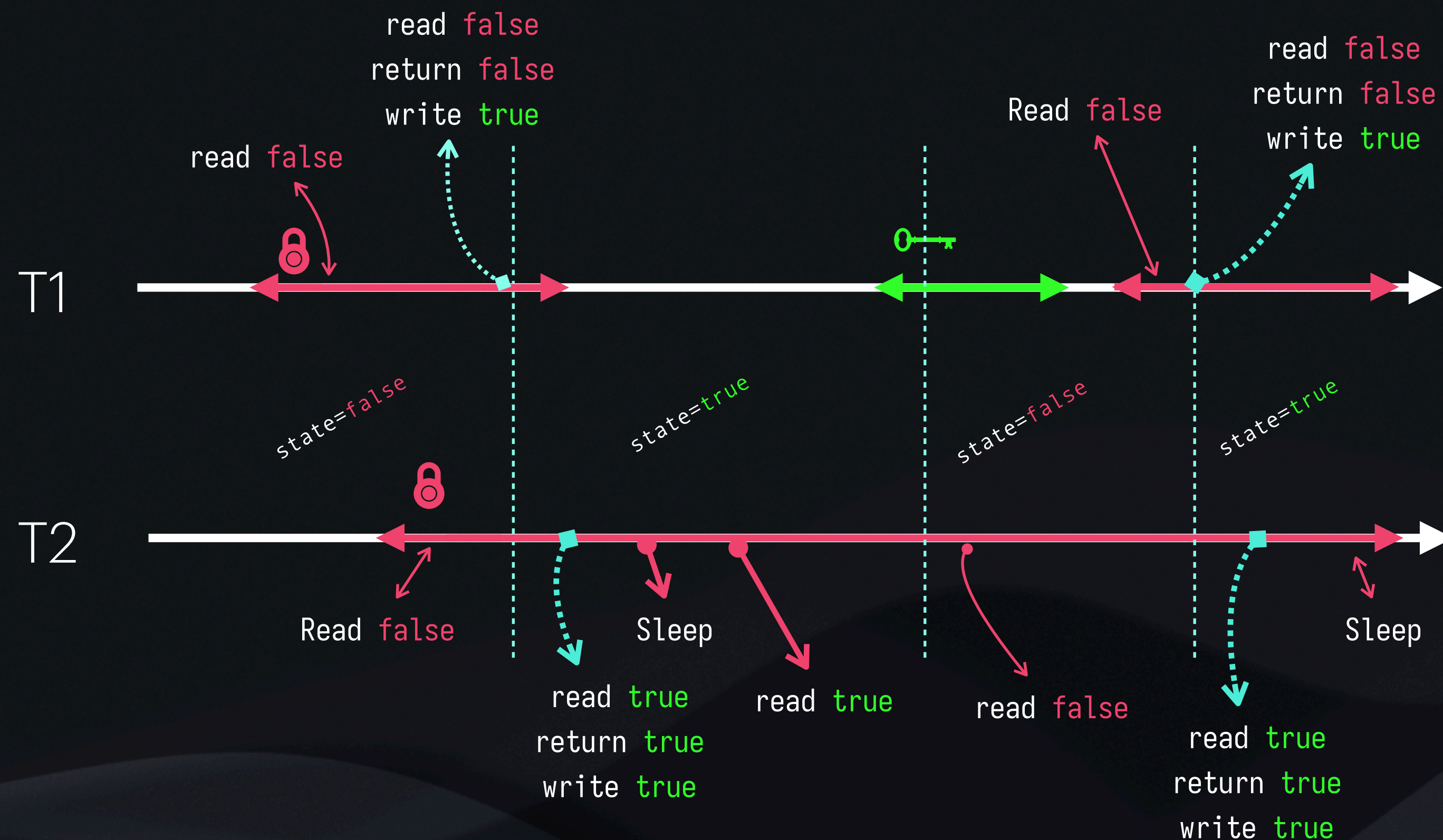

Exponential Backoff (BackoffLock)

- Cuando un hilo detecta que falló al adquirir un candado recién liberado, **eso indica** alta contención.
- En lugar de insistir inmediatamente, conviene **esperar un tiempo aleatorio** antes de reintentar.
- Esta espera (backoff) reduce el tráfico y da oportunidad a otros hilos.

```
1 public class Backoff {  
2     final int minDelay, maxDelay;  
3     int limit;  
4     final Random random;  
5     public Backoff(int min, int max) {  
6         minDelay = min;  
7         maxDelay = max;  
8         limit = minDelay;  
9         random = new Random();  
10    }  
11    public void backoff() throws InterruptedException {  
12        int delay = random.nextInt(limit);  
13        limit = Math.min(maxDelay, 2 * limit);  
14        Thread.sleep(delay);  
15    }  
16 }
```



Exponential Backoff (BackoffLock)



```

1 public class Backoff {
2     final int minDelay, maxDelay;
3     int limit;
4     final Random random;
5     public Backoff(int min, int max) {
6         minDelay = min;
7         maxDelay = min;
8         limit = minDelay;
9         random = new Random();
10    }

```

```

1 public class BackoffLock implements Lock {
2     private AtomicBoolean state = new AtomicBoolean(false);
3     private static final int MIN_DELAY = ...;
4     private static final int MAX_DELAY = ...;
5     public void lock() {
6         Backoff backoff = new Backoff(MIN_DELAY, MAX_DELAY);
7         while (true) {
8             while (state.get()) {};
9             if (!state.getAndSet(true)) {
10                return;
11            } else {
12                backoff.backoff();
13            }
14        }
15    }
16    public void unlock() {
17        state.set(false);
18    }
19    ...
20 }

```

El hilo T2, pierde dos veces el "get&Set()".

Exponential Backoff (BackoffLock)

- El **retroceso aleatorio** reduce el tráfico y mejora el rendimiento.
- Es una **solución sencilla pero efectiva** para situaciones de alta contención.
- Su **rendimiento depende** de la elección de parámetros (minDelay, maxDelay).
- **No es universalmente óptimo**, dependerá de la arquitectura, del sistema, número de procesadores/núcleos y sus velocidades.
- Sin embargo, **no garantiza fairness**: algunos hilos pueden tener más suerte en los tiempos de espera y adquirir el candado antes que otros.

Array-based Lock (ALock)

Los candados que hemos visto tienen algo en común, cuando múltiples hilos intentan adquirir el candado, todos leen y escriben en la misma dirección de memoria. Cuando el candado se libera, se generan invalidaciones de caché, lo que satura el bus del sistema.

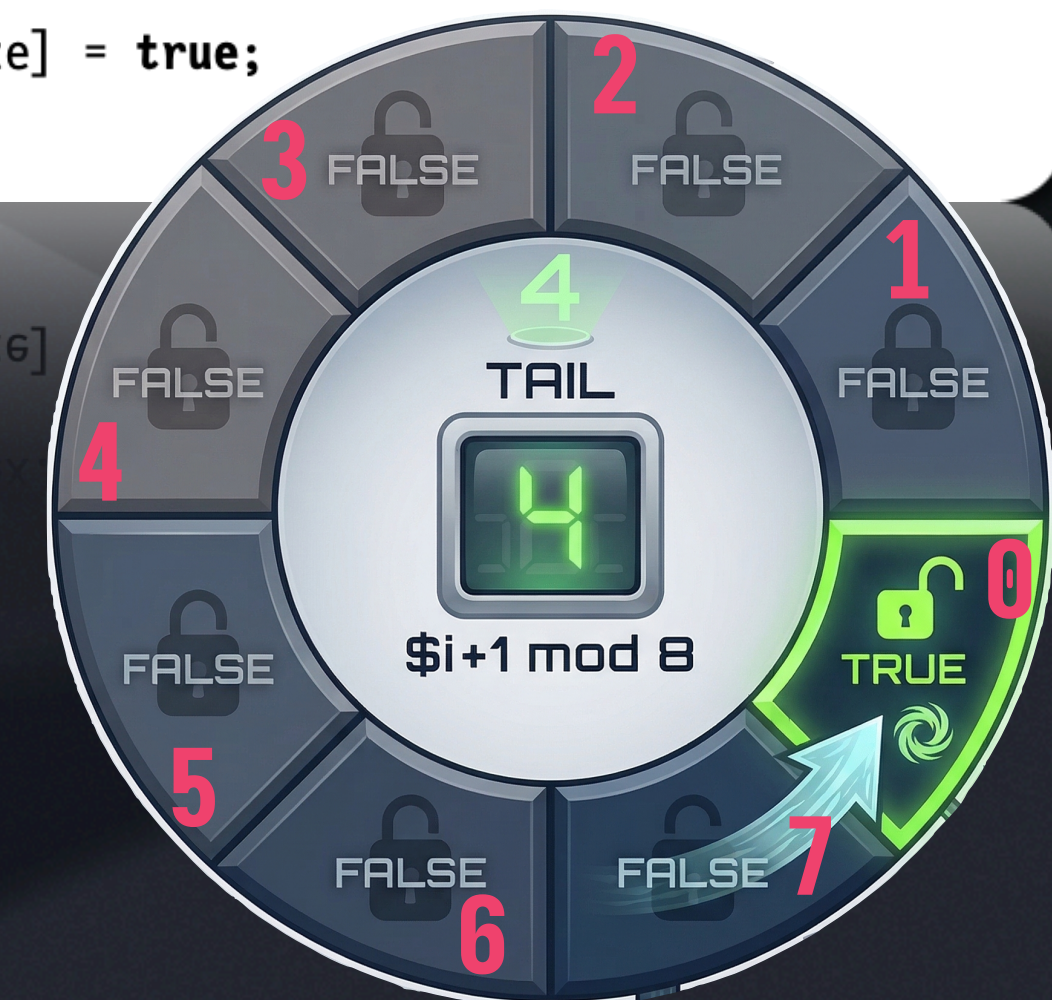
ALock() soluciona esto garantizando que cada hilo espere en una ubicación de memoria distinta.

- **Estructura de datos:** Arreglo booleano (**flag**) de tamaño **N** (**N** el máximo número de hilos que pueden competir por el candado).
- **Estado inicial:** Solo el primer elemento del arreglo es **true** (indicando que el candado está disponible para el primer hilo que llegue). El resto de los elementos son **false**.
- **Contador tail:** Variable atómica (entero), **tail** para llevar el registro del próximo espacio disponible en el arreglo.

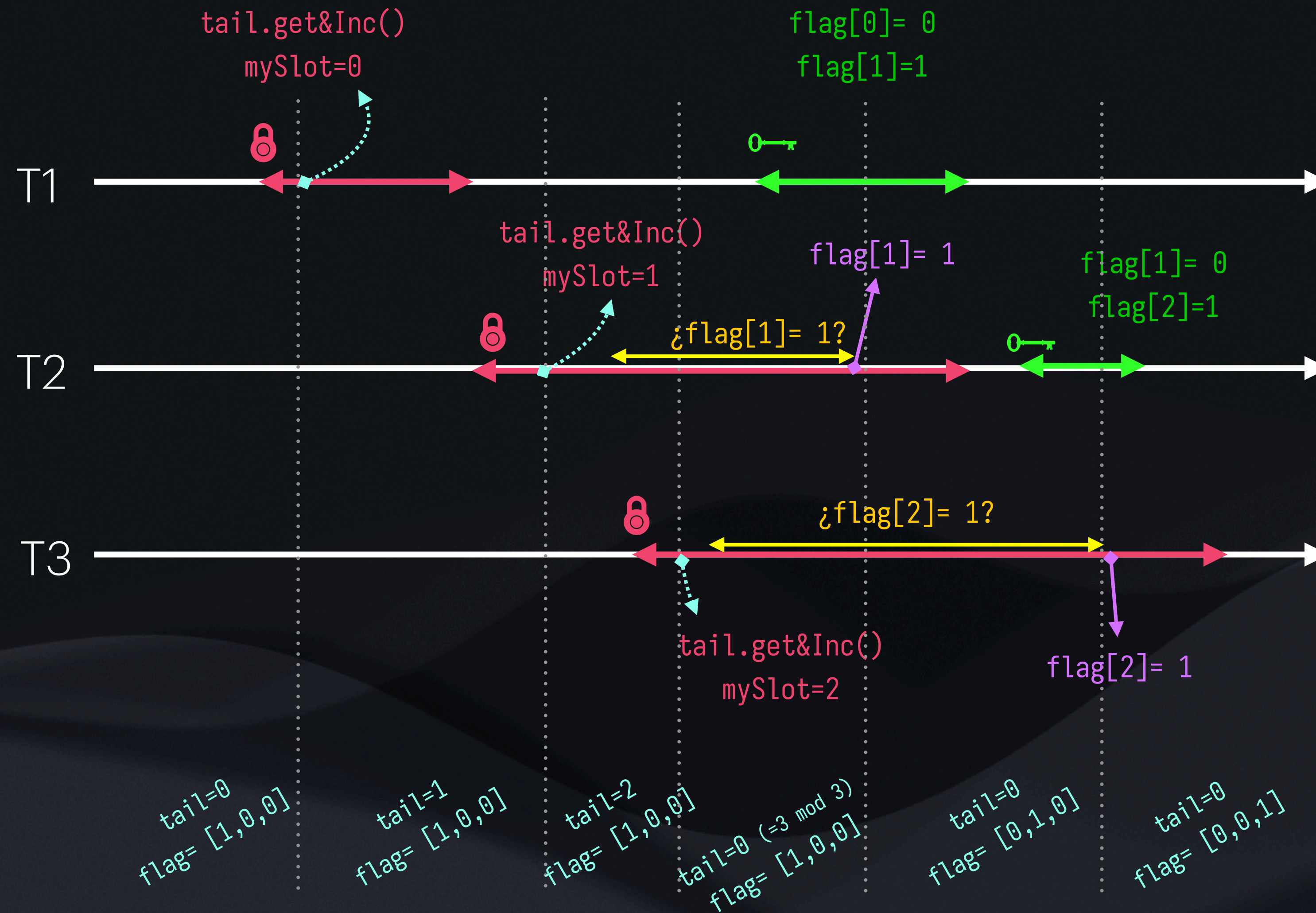
```

1 public class ALock implements Lock {
2     ThreadLocal<Integer> mySlotIndex = new ThreadLocal<Integer> () {
3         protected Integer initialValue() {
4             return 0;
5         }
6     };
7     AtomicInteger tail;
8     boolean[] flag;
9     int size;
10    public ALock(int capacity) {
11        size = capacity;
12        tail = new AtomicInteger(0);
13        flag = new boolean[capacity];
14        flag[0] = true;
15    }
16    public void lock() {
17        int slot = tail.getAndIncrement() % size;
18        mySlotIndex.set(slot);
19        while (! flag[slot]) {};
20    }
21    public void unlock() {
22        int slot = mySlotIndex.get();
23        flag[slot] = false;
24        flag[(slot + 1) % size] = true;
25    }
26 }

```



Array-based Lock (ALock)



```

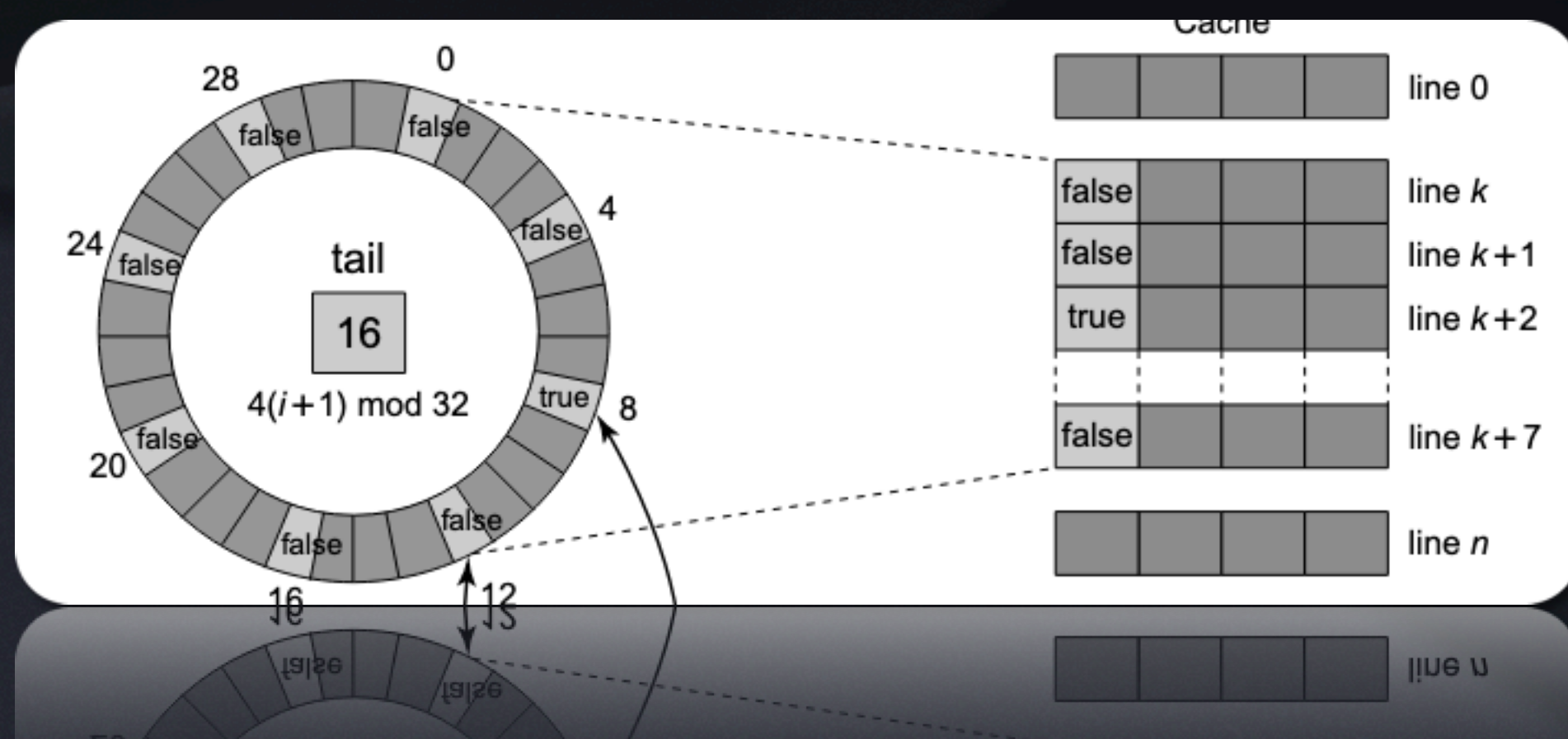
1 public class ALock implements Lock {
2     ThreadLocal<Integer> mySlotIndex = new ThreadLocal<Integer> () {
3         protected Integer initialValue() {
4             return 0;
5         }
6     };
7     AtomicInteger tail;
8     boolean[] flag;
9     int size;
10    public ALock(int capacity) {
11        size = capacity;
12        tail = new AtomicInteger(0);
13        flag = new boolean[capacity];
14        flag[0] = true;
15    }
16    public void lock() {
17        int slot = tail.getAndIncrement() % size;
18        mySlotIndex.set(slot);
19        while (! flag[slot]) {};
20    }
21    public void unlock() {
22        int slot = mySlotIndex.get();
23        flag[slot] = false;
24        flag[(slot + 1) % size] = true;
25    }
26 }

```


Array-based Lock (ALock)

1. La ventaja principal: Espera local

Aunque todo el arreglo `flag[]` es compartido por todos los hilos, el diseño logra minimizar el tráfico en la memoria. Esto se debe a que cada hilo hace spin (espera activa) leyendo únicamente su propia copia local guardada en la memoria caché del procesador, en lugar de saturar la memoria principal con consultas constantes.



2. El problema: *False Sharing*

A pesar de la ventaja anterior, el hardware real agrupa los datos de la memoria en bloques llamados líneas de caché (usualmente de 64 bytes).

- Como los elementos de un arreglo están pegados unos junto a otros en la memoria, varias banderas (booleanos) terminan agrupadas dentro de una misma línea de caché.
- Si un hilo modifica su bandera, el hardware se ve obligado a invalidar toda la línea de caché para los demás procesadores.
- Esto provoca que los otros hilos sufran invalidaciones y tráfico de red innecesario, incluso si estaban esperando banderas diferentes que no sufrieron cambios.

CLH Lock (Craig, Landin y Hagersten)

El algoritmo se basa en objetos de tipo `QNode`, los cuales tienen un único atributo booleano, `locked`.

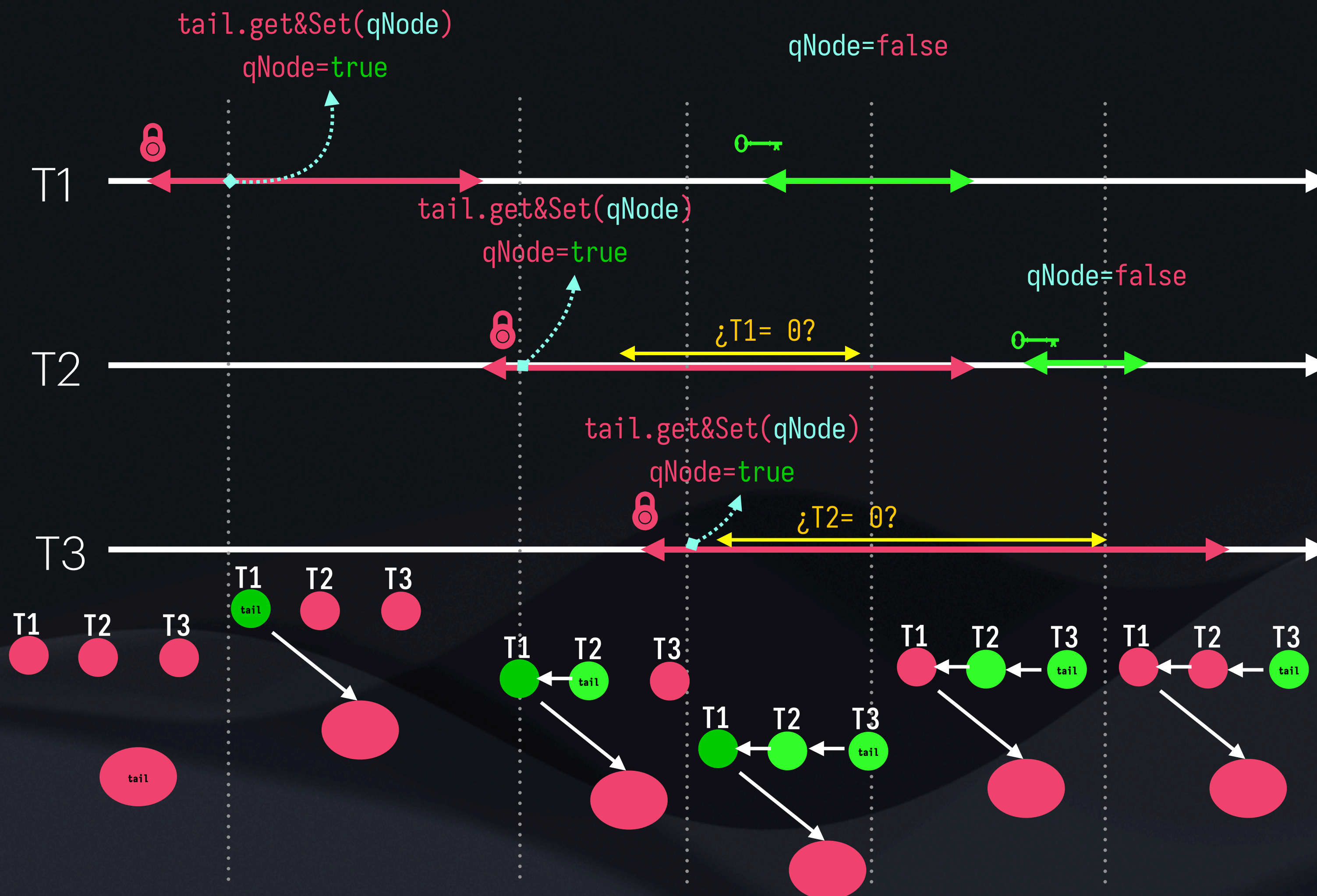
- `tail`: Es un puntero atómico global. Siempre apunta al último hilo que se formó en la fila. Se inicializa con un nodo "falso" o de arranque.
- `myNode`: Es una variable local al hilo (`ThreadLocal`). Representa el nodo actual del hilo.
- `myPred`: Es una variable local. Guarda una referencia al nodo del hilo que está justo delante de él en la fila.

```

1 public class CLHLock implements Lock {
2     AtomicReference<QNode> tail = new AtomicReference<QNode>(new QNode());
3     ThreadLocal<QNode> myPred;
4     ThreadLocal<QNode> myNode;
5     public CLHLock() {
6         tail = new AtomicReference<QNode>(new QNode());
7         myNode = new ThreadLocal<QNode>() {
8             protected QNode initialValue() {
9                 return new QNode();
10            }
11        };
12         myPred = new ThreadLocal<QNode>() {
13             protected QNode initialValue() {
14                 return null;
15            }
16        };
17     }
18     ...
19 }

```


CLH Lock (Craig, Landin y Hagersten)



```

20  public void lock() {
21      QNode qnode = myNode.get();
22      qnode.locked = true;
23      QNode pred = tail.getAndSet(qnode);
24      myPred.set(pred);
25      while (pred.locked) {}
26  }
27  public void unlock() {
28      QNode qnode = myNode.get();
29      qnode.locked = false;
30      myNode.set(myPred.get());
31  }
32  }

```

```

35  }
36  }
37  }

```